RELATÓRIO

Atividade 6 de E.D. – Tabela Hash

Componentes do grupo: Carlos Eduardo Souza Oliveira, Felipe Bernardes Freitas Sanches, Vinícius Cavalcanti Wagner.

A divisão das tarefas entre os membros do grupo foi feita da seguinte forma: Carlos foi o responsável por fazer o código da classe HashTable e da geração dos histogramas, Felipe fez a classe HashListaLigada e Vinicius fez as funcionalidades que envolvem o menu do programa. Além disso, Carlos escreveu o relatório e gravou o vídeo.

1 - Código

O programa foi feito completamente em python. Além da que estava nas instruções da tarefa, a classe HashTable possui outras duas funções hash, uma que faz o hash utilizando da primeira letra da palavra e outra que utiliza apenas a função nativa da linguagem, selecionadas por meio do método “escolher_hash”.

Adicionalmente, a classe também possui um método put e um get, os quais seguem as convenções apresentadas em sala, nas quais ao inserir uma chave repetida troca-se apenas o valor dela e não se deve permitir inserir elementos com chaves ou valores nulos. Ao der put ou get em um item, esses métodos também precisam de uma string que indica qual função hash deve ser considerada. Sempre que uma nova chave é adicionada, o método put incrementa o valor de n (quantidade de itens na tabela).

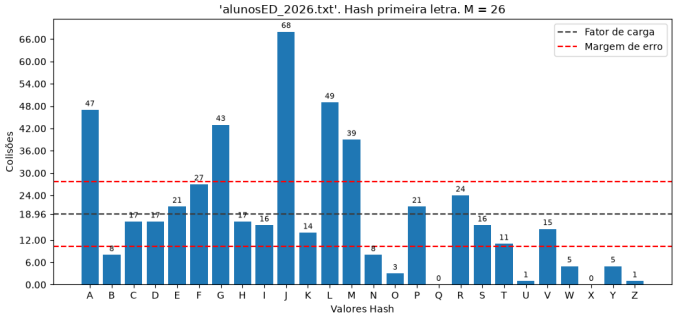
Cada “balde” não vazio da tabela contém uma lista ligada de “Node” com chaves e valores, a qual é responsável por armazenar os elementos do valor hash correspondente e contém um atributo que indica a quantidade de itens nela. A lista também possui métodos de inserção e acesso de itens, para poderem ser chamados pela classe da tabela hash, dessa forma o método de inserção da lista é o responsável por atender a convenção do que fazer ao se adicionar uma chave repetida.

Primeiramente carrega-se os nomes, utilizando de um parâmetro para dizer se deve-se ler por linha ou não, para caso deseja-se pegar cada linha ou cada palavra. O menu contém opções para a execução de cada questão da tarefa, além de um para se realizar um experimento com arquivo e M personalizados e uma das funções hash disponíveis no programa.

Em cada experimento, a tabela hash com o M correspondente é criada, depois a função “montar_HashTable” é chamada com essa tabela, a função hash desejada e com a lista de chaves que devem ser inseridas, provenientes dos arquivos de texto “alunosED_2026.txt” e “clara-dos-anjos.txt”. Como o valor não pode ser nulo e nem foram especificados, a função considera o index da chave para isso. Logo após, chama-se “gerar_histograma”, a qual recebe a tabela, um nome para o gráfico e um eixo x opcional. Ela também calcula o fator de carga e considera uma margem de erro para as quantidades de colisões seguindo a fórmula $\alpha \pm 2\sqrt{(\alpha)}$, selecionada com base no auxílio de ferramentes de IA, desenhando as linhas correspondentes no histograma e adequando a aparência do gráfico conforme o necessário. Em alguns dos experimentos o eixo x é o alfabeto, e quando não especificado ele é os valores hash que a tabela tem.

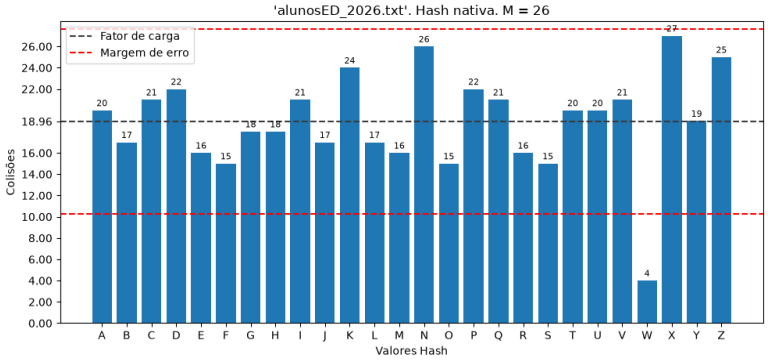
2 – Experimentos

1 – alunosED_2026, usando hash da primeira letra e M = 26.



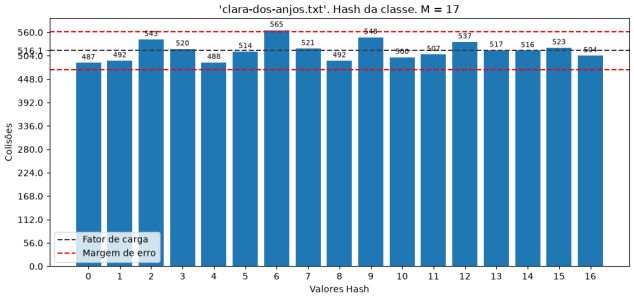
A função hash não é boa, pois as barras apresentam uma variação enorme, com muitas delas fora da margem de erro (14). O balde com a maior quantidade de colisões foi o J, com 68, e as menores foram Q e X, com 0, mas também U e Z tiveram apenas uma.

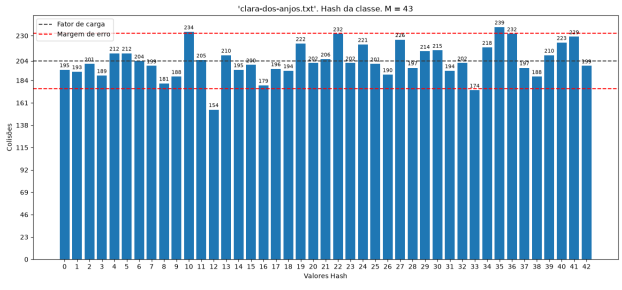
2 – alunosED_2026, usando hash nativa e M = 26.

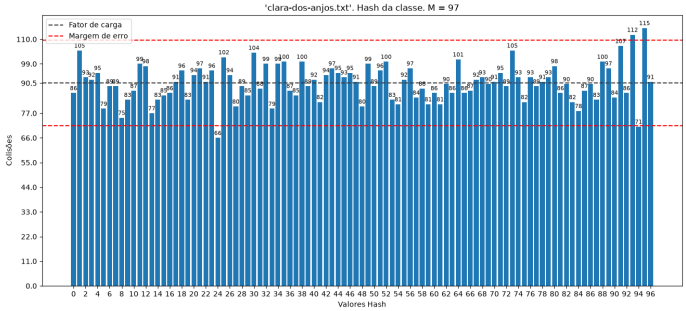


Como a hash nativa do python pode mudar com cada execução, esse exemplo não é constante e representa apenas uma delas. Considerando-se a margem de erro, essa função é bem melhor que a outra, pois apenas um balde se encontra fora do aceitável (W = 4). O balde com maior quantidade de colisões foi o X, com 27, e o com menor foi o W, com 4.

3 – clara-dos-anjos, usando hash da classe e M = 17, 43, 97 (primos).







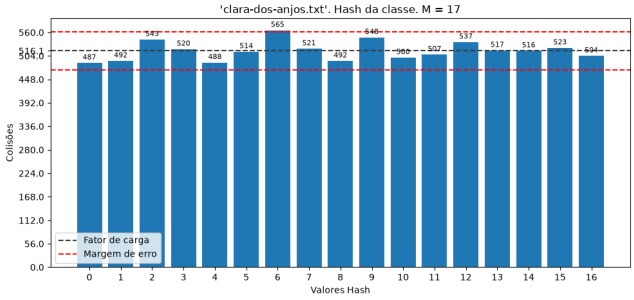
Entre esses experimentos, o com M = 17, foi o que aparentou uma melhor dispersão das chaves, pois ele é o que as barras estão mais dentros da margem de erro, com apenas uma (6), passando minimamente. A com M = 43 apresentou 4 valores fora da margem. Já a com M = 97 teve uma maior diferença de valores, apesar de apenas 4 estarem fora da margem.

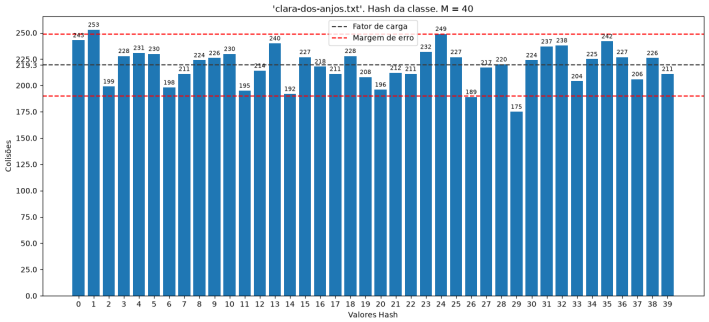
M = 17 – Maior colisão: hash 6, valor 565. Menor: hash 0, valor 487.

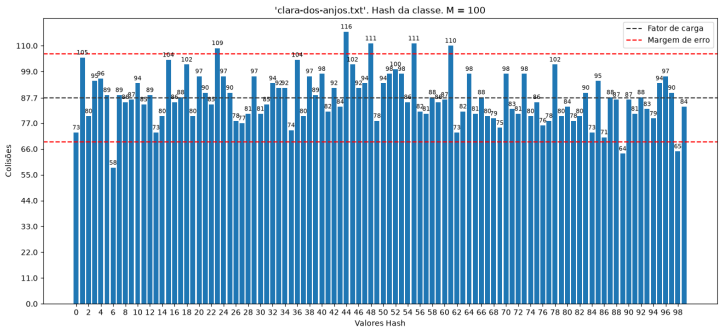
M = 43 – Maior colisão: hash 35, valor 239. Menor: hash 12, valor 154.

M = 97 – Maior colisão: hash 95, valor 115. Menor: hash 24, valor 66.

4 – clara-dos-anjos, usando hash da classe e M = 16, 40, 100 (não primos).







Esses experimentos tiveram resultados parecidos com os anteriores, já que usam a mesma função hash e uma pequena diferença nos valores de M.

Dessa forma, entre eles, o com M = 16, foi o que aparentou uma melhor dispersão das chaves, com apenas um minimamente fora do esperado. A com M = 40 apresentou 3 valores fora da margem. Já a com M = 100 teve uma maior diferença de valores, com 8 estando fora da margem.

M = 16 – Maior colisão: hash 7, valor 595. Menor: hash 13, valor 512.

M = 40 – Maior colisão: hash 1, valor 253. Menor: hash 29, valor 175.

M = 100 – Maior colisão: hash 41, valor 116. Menor: hash 5, valor 58.

Comparando-se os resultados dos experimentos 3 e 4, pode-se observar resultados muito parecidos, com, em termos de dispersão de valores, a de 16 e a de 17 sendo praticamente iguais, igualmente com as de 40 e 43, apesar da de 40 apresentar 1 valor a menos fora da margem, e com as de 97 e 100 apresentando a maior diferença, com a de 100 tendo o dobro de valores fora da margem e uma pior dispersão. Disso pode-se notar que, para a hash escolhida, maiores valores de M se saem gradativamente piores, além de também ser bem mais perceptível a diferença de eficiência entre Ms primos ou não.